

PROJECT AND TEAM INFORMATION

Project Title

Secure CBT: Hierarchical Key-Chained Question Paper Delivery & Leak-Traceability System

Student / Team Information

<i>Team Name: Secure CBT</i> <i>Team #: DSCPP-III-2026-T145</i> <i>Semester: 3rd Section: BTECH</i> <i>Mentor: Mr. Kartikey Arora</i>	
Team member 1 (Team Lead) <i>(Last Name, name: student ID: email, picture):</i>	<i>Naman –</i> <i>2510013438</i> <i>2510013438@geu.ac.in</i> 
Team member 2 <i>(Last Name, name: student ID: email, picture):</i>	<i>Gupta, Aayush –</i> <i>2510370185</i> <i>2510370185@geu.ac.in</i> 
Team member 3 <i>(Last Name, name: student ID: email, picture):</i>	<i>Bhushan, Kushagra –</i> <i>2510012263</i> <i>2510012263@geu.ac.in</i> 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

Question-paper leaks keep showing up in India's education and recruitment exams, and question-paper leaks have been happening for years. Boards at both the state and national level have had to deal with question-paper leaks at some point – results get held up. In a fair number of cases the exam simply has to be conducted again. A delayed or cancelled exam pushes back when qualified candidates actually start their education or their jobs, sometimes drags on into litigation, and puts real stress on students who are just waiting to hear whether their exam even counted. There is also a quieter cost: every question-paper leak makes people trust the system a little less, and in a country where a single exam can decide someone's entire trajectory, that erosion of trust matters. We are not trying to investigate any question-paper leak or assign blame here – that is not really our project's job. Instead we are treating this purely as a systems-security problem: can a Computer-Based Test (CBT) pipeline be built so that leaking is hard, by design, and if a question-paper leak does slip through anyway can the system point to where it happened? Our Secure CBT system tries to answer that by locking down the three places a paper is actually vulnerable – when the paper is set, when the paper is distributed, and at the exam centre itself – using a uniquely keyed encryption scheme.

State of the Art / Current solution (1 pt)

It is not the case that nothing is being done today. CBT systems – the ones that the National Testing Agency runs for JEE, NEET, and CUET – already stack layers of security on top of each other:

- **Encrypted storage** – the paper sits encrypted on a central server instead of as a plain file that anyone could open.
- **Candidate authentication** – requires OTP verification at the centre before the paper unlocks.
- **Time-locked decryption** – after the encrypted file reaches the centre, it will not decrypt before the scheduled slot.
- **Physical safeguards for offline exams** – papers are printed at a small number of approved presses and moved around in tamper-evident packets.

All of these measures do cut down on opportunistic leaks. However, there is a gap that all of these approaches share: everyone receives the same encrypted file. If one copy leaks, the ciphertext itself tells you nothing – it looks identical to every centre's copy. Determining where the leaked encrypted file actually came from means digging through access logs, attendance records, duty rosters, or whatever paper trail exists after the fact. That task is genuinely hard to get right – one analysis of exam-fraud cases in India found 148 incidents over an 11-year span, and only a single one ended in a conviction [7]. So the gap we are targeting is not that there is no security – there clearly is security. The problem is that none of it is built to trace the encrypted file back to a node. That is the piece our hierarchical, uniquely-keyed encryption scheme is meant to add, baked directly into the ciphertext rather than reconstructed afterward.

Project Goals and Milestones (2 pts)

The main aim is a CBT question-paper system that does three things all at once. It stops the paper from getting out during each of its three vulnerable points. It makes sure every person taking the test gets the same set of questions. If a leak still happens, it finds exactly where it came from – all the way down to the specific centre or student terminal – because of how the system is built, not because someone goes looking for it.

Prevention works by spreading the work out so no single person has all the knowledge. It uses encryption that is compressed with Huffman coding and has keys organized in a hierarchy, so each node ends up with its own uniquely encrypted version of the paper. It also uses time-locked decryption at the centre.

Fairness is guaranteed because every node ends up with the identical set of questions once unlocked.

Traceability is guaranteed because each node's encrypted version comes from a chain of parent keys and its own secret – there is no need for a separate fingerprint database.

#	Milestone	Status
1	Finalize hierarchical key-chaining scheme and class/data-structure design for all three phases	Achieved
2	Implement QuestionBank, Teacher, and PaperGenerator classes for distributed paper setting	Achieved
3	Implement Huffman compression of the assembled paper prior to encryption	In Progress
4	Build LCRS binary-tree representation of the server hierarchy and BFS broadcast routine	Pending
5	Build AVL-tree node registry for $O(\log n)$ lookup of node secrets and status	Pending
6	Implement hierarchical XOR key-chaining and AES encryption at every node	Pending
7	Implement SHA-256 hash-chained transfer-integrity log	Pending
8	Implement time-locked decryption at the centre/terminal level	Pending
9	Add Dijkstra-based route selection and MST-based backbone design (graph-layer enhancements)	Pending
10	Integrate all modules into one end-to-end CBT simulation	Pending
11	Evaluate leak-source tracing accuracy, tamper detection; prepare final report and demonstration	Pending

Project Approach (3 pts)

Phase 1 — Paper Setting

- Teachers independently enter their questions in plain text onto the trusted Central Server alone; no single teacher is allowed to carry out the final assembly.
- The PaperGenerator chooses N questions by means of random selection based on specified criteria (such as topic, difficulty, and marks-weightage) in order to ensure that the paper remains balanced, before converting the collected set into a single byte stream.
- The paper, once converted in this way, is compressed using the Huffman coding algorithm, which involves constructing a Huffman tree driven by a min-heap to create a prefix-code table that replaces common byte patterns with shorter codes.
- Compression takes place before any encryption, so each subsequent copy of the file becomes smaller — this point is important since the file is duplicated at every node in Phase 2.

Phase 2 — Distribution

- The compressed paper is encrypted at every node using an XOR key-chaining method. To decrypt data at any level you need both the parent node's key and the node's own secret – that means no level can be skipped or bypassed.
- Keys and ciphertexts are created on the fly exactly when they are needed. Each node keeps only a small secret value; nothing is stored in bulk ahead of time. This keeps the storage overhead steady even as the hierarchy grows wider.
- The actual structure of the distribution is an N-ary tree. In code it is implemented using the Left-Child Right-Sibling technique: each node has a `firstChild` pointer and a `nextSibling` pointer, which lets us build a tree with any number of children using two pointers per node, making it a strictly binary structure.
- The paper flows down the tree in a breadth-first manner – all nodes at one level are processed before moving to the next. The algorithm only moves from parent to child, never up, so back-flow is not possible by design.
- A separate AVL tree indexes every node by ID to help find nodes quickly. This tree is balanced and guarantees $O(\log n)$ lookup, and it holds the secret, assigned key, and current status for each node.
- Every time the paper moves from one node to another, the action is recorded in a hash-chained linked list. Each entry includes a SHA-256 hash of the previous entry, making the chain secure – if anyone tampers with any part of the chain, the hash breaks, making tampering immediately obvious.
- As an added feature, a weighted graph represents the real physical network. Dijkstra's Algorithm runs on this graph to find the safest route between nodes that are adjacent in the hierarchy, ensuring the data takes a secure path when it moves.
- Also, a Minimum Spanning Tree is used to create the lowest-cost backbone connecting all State servers in the most efficient way, saving bandwidth and reducing overall cost.

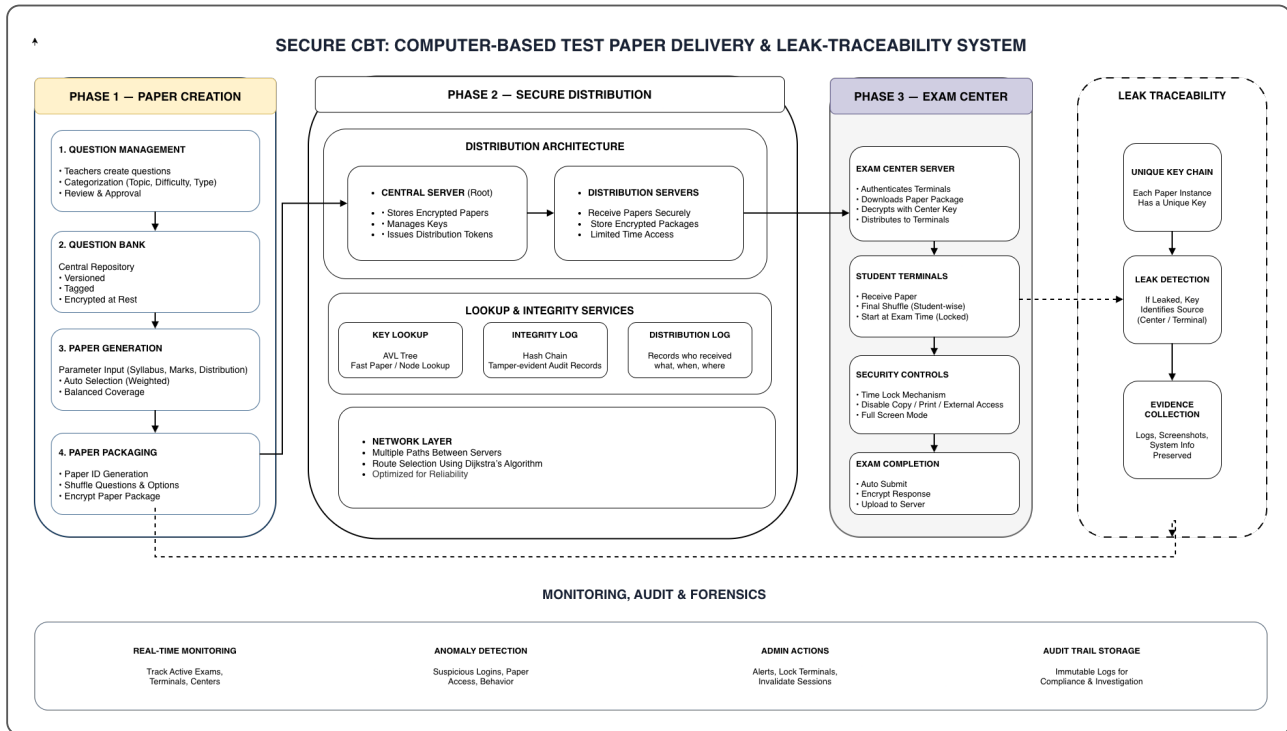
Phase 3 — At the Exam Centre

- Because each node's ciphertext already has a unique key, we do not need a separate per-student reordering step or a fingerprint hash map at this point.
- The Centre keeps its encrypted copy but cannot decrypt or publish that copy to the student portal before the scheduled exam time. This restriction is enforced by a time check, or by using a min-heap Extract-Min when a Centre manages several exam slots.
- If any encrypted file appears publicly, that file is matched directly against the AVL registry to find the Centre or terminal to which it was issued.

Platforms & Technologies

- **C++** — core implementation of all data structures and algorithms.
- **OpenSSL** (or an equivalent crypto library) — AES-256 for file encryption, SHA-256 for the hash-chain.
- **Git/GitHub** — version control and collaborative development.

System Architecture (High Level Diagram)(2 pts)



Architecture diagram rendered from the supplied draw.io (synopsisarch.pdf).

Project Outcome / Deliverables (1 pts)

By the end of this project we hope to have a working CBT question-paper security system that deals with all three stages where leaks can happen, from start to finish. The system will have traceability built directly into the encryption process instead of adding it later as a separate logging system. Concretely, we will provide:

- **Complete C++ source code** that includes the QuestionBank and PaperGenerator classes, Huffman compression, the LCRS binary-tree structure with BFS broadcast, the AVL node registry, hierarchical XOR key-chained AES encryption, the SHA-256 hash-chained audit log, Dijkstra-based route selection, MST backbone design, and time-locked decryption.
- A **test run across multiple states, multiple districts, and multiple centres** to show that the system works properly from one end to the other.
- A **tamper-detection demonstration** that proves a damaged transfer is caught right away by the hash-chain.
- A **leak-tracing demonstration** that finds where a leak happened from a simulated leaked ciphertext, using only the AVL registry.

Along with the code, we will create documentation that explains the system design, class diagrams, the data structures and algorithms used at every step, and the results of our testing.

Assumptions

The system expects that the network connection between levels is dependable when sharing information. It also expects that each node has its identity and secret information set up securely before the exam cycle starts (e.g. during centre registration), so that secret values are never sent together with the encrypted paper. The system also expects that the main server and its root key are not compromised, since the whole chain of secrets depends on this trusted key; if the main server were compromised, that would need to be fixed by rotating keys, which is outside what this project's simulation is trying to show. Finally, all devices are expected to have their clocks reasonably synchronized so that the time-based unlocking process can work correctly.

RESOURCES REQUIRED

- Development machines with a C++17 (or later) compiler and standard build tools.
- A cryptography library (e.g. OpenSSL) for AES-256 and SHA-256 primitives.
- Git/GitHub for version control and team collaboration.
- Sample question datasets across multiple subjects/topics for realistic testing.
- Access to a code editor/IDE and standard debugging/profiling tools.

References

1. Newslaundry, “10 years, 89 paper leak cases, 48 retests: From centre to states, few plugs for a leaky record” (2024) — data journalism piece compiling paper-leak incidents across Indian exam boards.
2. Wikipedia, “List of paper leaks in India” — consolidated timeline of reported paper-leak incidents.
3. The Wire, “India’s Exam Fraud Bubble: 148 Cases, One Conviction in 11 Years” — independent data analysis of Indian exam-fraud incidents, 2015–2026.
4. Cormen, Leiserson, Rivest, Stein — Introduction to Algorithms, MIT Press (reference text for Huffman coding, BFS, Dijkstra’s Algorithm, and Minimum Spanning Tree constructions).
5. NIST FIPS 197 — Advanced Encryption Standard (AES).
6. NIST FIPS 180-4 — Secure Hash Standard (SHS), specifying SHA-256.